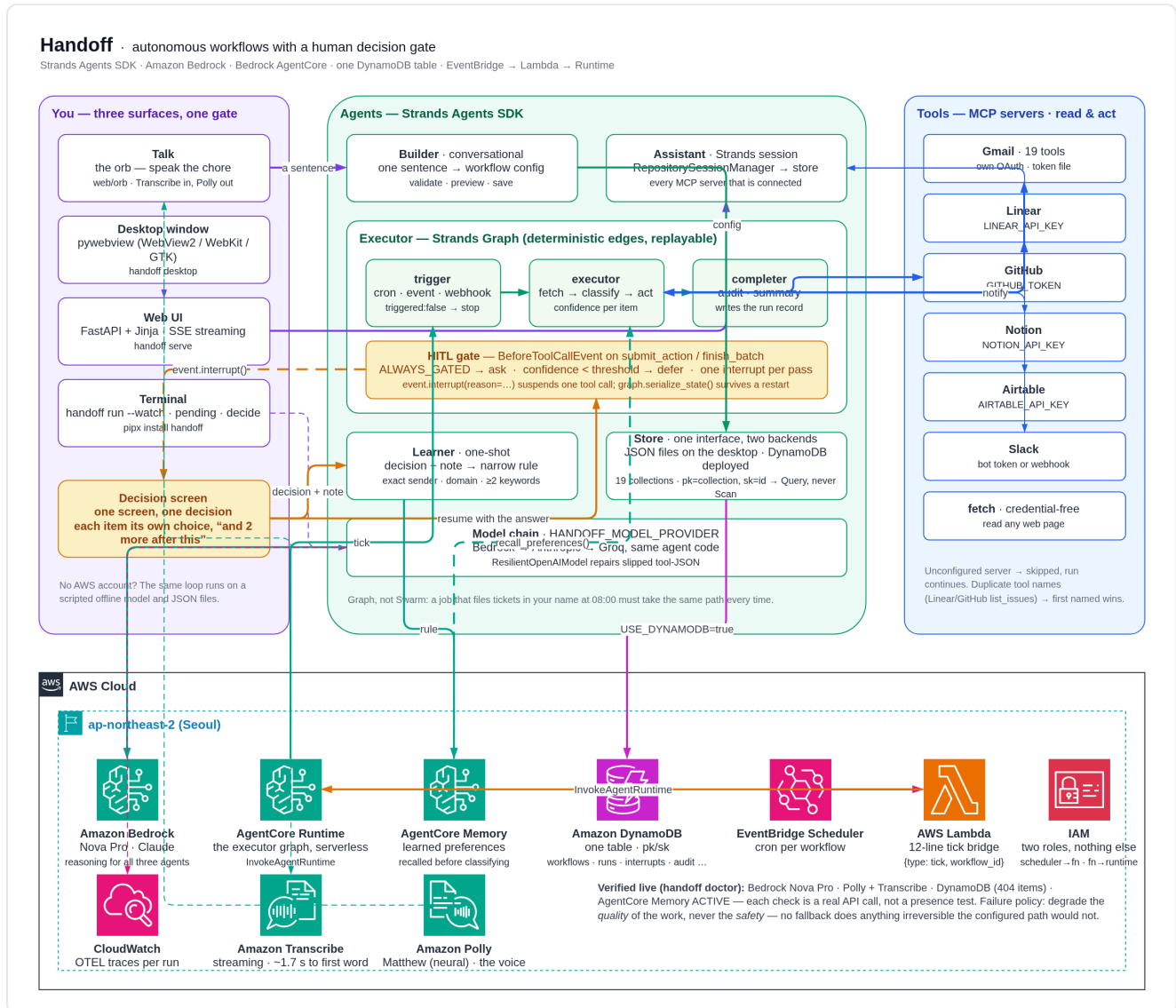


Handoff — architecture

Describe it. Hand it off. It runs. — autonomous workflows with a human decision gate, built on the Strands Agents SDK and Amazon Bedrock AgentCore.



Editable source: docs/architecture.drawio (draw.io, official AWS Architecture Icons) · generated by scripts/make_drawio_architecture.py

AGENTS

Builder · Executor (Strands Graph) · Learner · Assistant

GATE

BeforeToolCallEvent → event.interrupt(); one interrupt per pass

REASONING

Amazon Bedrock (Nova Pro / Claude) · Anthropic · Groq

STATE

one DynamoDB table (pk = collection, sk = id) or JSON files

SCHEDULE

EventBridge Scheduler → Lambda → AgentCore Runtime

VOICE

Amazon Transcribe in · Amazon Polly out

The shape of the problem

A background agent has exactly one hard question: *when is it allowed to act without asking?*

Answer "always" and you have something that files tickets from spam and archives the email you needed. Answer "never" and you have a worse inbox with extra steps — a notification for every decision you were already making yourself.

Everything in Handoff's design falls out of answering that question well, and of being able to *change* the answer as the system learns what you actually want.

System

The interrupt gate

Where it sits

On `BeforeToolCallEvent`, in front of `submit_action` — the only tool in the executor that changes anything outside the process. That placement is the whole design:

- **Before, not after.** The hook fires before the tool body runs. When the gate stops a call, nothing has happened to your mail. A gate that fires after the fact is an audit log, not a gate.
- **One chokepoint, not many.** Fetching mail and classifying it are free and reversible; only the action tool is gated. Gating the read path would produce interruptions with no safety value.
- **On the executor node only.** The trigger and completer nodes are never in a position to do something irreversible.

What it decides

<code>tool in ALWAYS_GATED</code>	→ ask, regardless of confidence (irreversible: delete, send)
<code>tool in CONDITIONALLY_GATED</code>	→ ask if <code>force_interrupt</code> , or <code>confidence < threshold</code>
<code>anything else</code>	→ never ask

Confidence is reported by the model itself, and the executor's system prompt is explicit that inflating it to avoid interruption and deflating it to avoid responsibility are both failure modes. A malformed score is treated as zero — failing toward asking, never toward acting.

How it suspends and resumes

```
response = event.interrupt(  
    interrupt_name,  
    reason=payload.model_dump(mode="json"),  
)
```

This one line does two different things depending on when it runs:

Pass	Behaviour
First	Raises <code>InterruptException</code> , unwinding the agent loop. The run returns with <code>stop_reason = "interrupt"</code> and the pending interrupt attached.
After resume	Returns the human's answer. Execution continues into the tool.

So the gate isn't a branch in the graph — it's a *suspension of one tool call*. The agent's message history, its partial reasoning, and the rest of the batch it was working through are all still there when the human answers.

Resume happens by re-invoking with an interrupt-response content block:

```
agent([{"interruptResponse": {"interruptId": interrupt.id, "response": {...}}])
```

Applying the answer

The hook rewrites the pending call rather than letting the original one through:

- A chosen action becomes `tool_use["input"]["action"]`, with confidence forced to 1.0 and `decided_by` stamped as human.
- "Leave it" sets `event.cancel_tool`, so the tool never runs at all. This matters for the audit trail: *declined* and *executed a no-op* are different facts, and a log that conflates them can't answer "what did it actually do".

One pass, one screen: the batch interrupt

The first live runs on an open-weight model exposed a product flaw in the naive design. With one tool call per item, the *first* unsure item suspended the loop — items five through eight never got looked at until the person answered, and they got one interruption per question. The whole point was to not be interrupted.

So an unsure `submit_action` is **deferred**, not interrupted: the gate records the payload in the run context, the tool body sees the deferral and does nothing to the item, and the executor keeps going. When the pass is complete it calls `finish_batch`, and *that* hook raises a single `event.interrupt()` whose reason carries every deferred item:

```
response = event.interrupt(BATCH_INTERRUPT, reason={"count": n, "items": [...]}))
```

The human answers on one screen (each item its own decision, "and 2 more after this"), the run resumes once all are answered, and `finish_batch`'s body carries out the chosen actions. Same Strands primitive, same serialised graph state, same "this run picked up where it left off" — one interruption instead of three. Measured on a real model: 8 items, 7 handled alone, 1 escalated, one interrupt, 3m47s end to end.

Immediate mode still exists (`HITLGate(batch=False)`) and is what the unit tests exercise; an embedded host uses it too, because a host that prompts inline has no reason to defer.

Surviving a restart

A run paused at 08:04 and answered at 11:30 might span a daemon restart, so `WorkflowRunner._settle` persists `graph.serialize_state()` onto the run record and `resume()` rehydrates it. Handoff keeps no parallel pause state of its own — the Graph *is* the state.

One subtlety worth knowing: resuming replays the gated tool calls, so `resume()` pre-seeds the gate with **every** interrupt the run has already raised, not just the one being answered. Without that, the still-pending questions would be re-raised under fresh ids, orphaning the links the UI holds and re-notifying the user about questions they've already seen.

Three agents

Separated because the jobs fail differently, not for tidiness.

	Builder	Executor	Learner
Shape	conversational	autonomous graph	one-shot
Runs	while you type	on a schedule	after each decision
Fails by	misunderstanding the ask	acting when it shouldn't	over-generalising
Guardrail	preview before save	the interrupt gate	narrow matching

Collapsing them into one agent would mean one system prompt trying to hold three different notions of caution at once.

Why Graph, not Swarm

A workflow is a fixed, auditable sequence. Graph gives deterministic edges, one entry point, and a replayable execution order. Swarm gives open-ended collaboration, which would make every run take a different path through the same job. For something that files tickets in your name at 8am, "different every time" is a bug.

The trigger → executor edge carries a condition: if the trigger node reports triggered: false — an empty webhook, say — the executor never runs. An empty signal shouldn't cost a model invocation, let alone a ticket.

The learning loop

```
interrupt → human answers → Learner writes a rule → stored
                                     ↓
next run: recall_preferences() → classify_email() applies it
          → confidence rises above threshold → handled silently
```

Measured in the demo: run one asks three questions, run two asks none, and the three items are recorded as decided_by: memory so the dashboard can show *interruptions that didn't happen* as a distinct number from *things it was always confident about*.

Restraint is the hard part

A rule that fires too eagerly takes actions nobody sanctioned — strictly worse than one extra question. So find_matching_preference will only fire on:

1. an **exact sender** match, or
2. a whole **domain** (@vendor.com), or
3. **at least two distinct keyword hits** — one shared word is a coincidence.

And the Learner's prompt pushes toward narrow rules, treats the user's free-text note as the strongest available signal, and instructs it to scope tightly rather than invent a broad rule when it can't see a generalisable one.

Providers, and surviving cheap ones

Bedrock, Anthropic and Groq sit behind one switch (HANDOFF_MODEL_PROVIDER); the agent code is identical. Groq matters because it needs one key, has a free tier, and serves the speech models that make voice real — so a fresh clone can reason and talk today, and move to Bedrock/AgentCore for the deployment.

Two things learned running open-weight models through Strands:

- **Tool-call JSON slips.** gpt-oss on Groq systematically emitted "confidence": 0.95" — a stray quote — and Groq's endpoint rejects the whole response with tool_use_failed. Re-sampling at low temperature reproduces the same slip. But the endpoint hands the broken text back as failed_generation, and the judgement inside it is correct. So ResilientOpenAIModel repairs the JSON and emits the call as if the model had produced it cleanly; re-sampling is the fallback. Only in non-streaming mode, where nothing has reached the agent yet when the failure surfaces.

- **Tokens are the budget.** Free tiers meter tokens per minute and per day. Every tool argument the model re-types — a subject line, a preview — costs on every subsequent turn. So tools take ids and judgement only; the run context holds the text. Context per call fell from ~5K to ~2K tokens.

Storage

Five collections — workflows, runs, interrupts, audit, preferences — behind one `Store` interface with two backends:

- **JSON files** (default). The whole product runs on a laptop with no AWS account, which matters when someone wants to clone the repo and press play.
- **DynamoDB** (`USE_DYNAMODB=true`). Same models, same methods.

Access patterns are "get one by id" and "scan a small table", which is all the dashboard and the demo need.

Two front ends, one gate

Surface	The question goes to	Interrupt behaviour
Web / desktop (<code>web/server.py</code>)	FastAPI decision screen	escapes; answered out of band; run resumed later
Embedded (<code>host/gate.py</code>)	the host's <code>request_human_input</code>	answered inline; the loop never unwinds

`HostGate` subclasses `HITLGate` and passes the human's answer to `event.interrupt(..., response=answer)` as a *preemptive* response — from the agent's side, the question was already answered when it asked.

If the human channel is unreachable, `HostGate.ask` returns `skip`. An agent that cannot ask must not decide to act anyway.

Failure behaviour

When	It does
Confidence is malformed	treats it as 0 — asks
The human channel is down	skips the item, never acts unilaterally
Slack/SNS notification fails	logs to stdout; the run still completes
An MCP server is unconfigured	skips that integration, runs with the rest
AgentCore Memory is unreachable	falls back to the local preference store
AgentCore Browser is unavailable	falls back to a plain HTTP fetch
No AWS credentials at all	the scripted offline model runs the full loop

The pattern: degrade the *quality* of the work, never the *safety* of it. Every fallback either does less or does it worse — none of them does something irreversible that the configured path wouldn't have.

The v2 platform layer

Everything above is the engine. v2 adds the studio around it, on the same store and the same SDK.

One store, one table

Fourteen collections (workflows, runs, interrupts, audit, preferences, workspaces, credentials, tool servers, skills, agents, memory, schedules, artifacts, sessions, usage, chats, chat messages, agent runs, profile) share one interface with two backends. On the desktop it is JSON files under the state directory. Deployed it is **one DynamoDB table** with a composite key — pk is the collection name, sk the item id — so listing a collection is a Query, never a Scan, and one collection's rows can never be handed to another's model. The earlier three-table layout could not work: each collection has its own id attribute and a table has exactly one key schema.

Chats are Strands sessions

A chat is a Strands Agent with a RepositorySessionManager whose SessionRepository is backed by that store ([chat/repository.py](#)). Messages are rows; the session id is the chat id. A conversation survives a restart and, deployed, lives in DynamoDB — the same durability as a run. The agent's tools are the workspace's levers (run_workflow_now, recent_runs, pending_decisions, decide, memory, artifacts) plus the Builder's, plus every enabled MCP server whose credentials are present.

A turn runs on a worker thread and narrates itself over the in-process events bus: delta for text, tool_start / tool_end from a HookProvider ON BeforeToolCallEvent / AfterToolCallEvent, asked

when the turn left decisions waiting, done with the final text, any workflow config it produced, and token usage. The page paints from SSE. The same narrator drives the agent workbench.

The interrupt reaches the chat

A workflow started from chat runs the normal graph with the normal gate. When it stops, the decision exists in the store like any other; the turn diffs pending interrupts before and after and emits `asked`, and the chat page fetches the same decision card Activity renders. Answering it in either place resumes the same run.

`workspace.yaml`

[platform/workspace_yaml.py](#) renders a workspace as a document — workflows, non-built-in skills, agents, tool servers; never credentials — and applies one back with *the file is the truth* semantics: things named are created or updated, things the workspace had but the file no longer lists are removed. A bundle is that file plus one Markdown per skill and one JSON per agent, importable anywhere, from the UI or `handoff workspace import`.

Cloud cron

EventBridge Scheduler cannot target AgentCore Runtime. The schedule fires a twelve-line Lambda that forwards its payload — the same `{"type": "tick", "workflow_id": ...}` the runtime accepts from anything else — with `InvokeAgentRuntime`. Two IAM roles: one the scheduler assumes (may invoke the function), one the function assumes (may invoke the runtime). Nothing else may.

The shell

A token-based design system: `tokens.css` (spacing, type, `light-dark()` colour pairs), `reset.css`, `ui.css` (blocks with modifiers) and `pages.css` (what knows a workflow from a run). An inline SVG sprite for icons. The sidebar is data ([web/nav.py](#)) — global tools, then Discover, then workspaces with a sub-nav that unfolds under the active one. `app.js` owns theme, palette, toasts, hotkeys and the live status dot; `chat.js` owns streaming; `handoff.js` owns voice.

Appendix — verified live

Output of `handoff doctor` on the submission machine. Every row is a real API call against the service, not a check that a variable is set.

Status	Service	Result
PASS	Groq API	qwen/qwen3.8-27b — free tier, tool-JSON repair
PASS	AWS Bedrock	apac.amazon.nova-pro-v1:0 in ap-northeast-2
PASS	Speech	Polly Matthew (neural) + Transcribe streaming
PASS	Gmail	signed in — 19 tools live (own OAuth)
PASS	Linear	Arpit Singh
PASS	Slack	bot in slack_hackathon_test
PASS	GitHub	@LSUDOKO
PASS	Notion	workspace HandOff-Amazon
PASS	Airtable	personal access token
PASS	DynamoDB	table handoff (404 items)
PASS	AgentCore Memory	handoff_preferences (ACTIVE)

11 working, 0 broken, 1 not configured (Anthropic API key not set — Bedrock is the active provider).